# UNIVERSITY ADMISSION DECISION ENGINE

Project Report

## 1. Problem Statement

Develop a Python program that evaluates university applicants using subject eligibility, aggregate marks, course-specific thresholds, and tie-breaking criteria, then generates a ranked list of eligible applicants.

## 2. Objectives

• Check whether each applicant satisfies required subject marks.

• Compare the applicant's aggregate marks with the course-specific threshold.

• Identify eligible and non-eligible applicants.

• Rank eligible applicants using aggregate marks.

• Use entrance score and applicant name as tie-breaking criteria when needed.

• Display a clear ranked list of eligible applicants.

## 3. Assumptions

• Each applicant has marks for the required subjects.

• Every course has a minimum aggregate threshold.

• The required subject minimum is 60 marks.

• The general aggregate eligibility is 70%.

• Higher aggregate gets the higher rank.

• If aggregate is tied, higher entrance score gets the higher rank.

• If both are tied, applicant name is used as the final tie-break.

## 4. Algorithm

1. Start.

2. Read applicant details, subject marks, aggregate, entrance score, course, and course threshold.

3. Check required subject marks.

4. Check aggregate marks and course-specific threshold.

5. If all conditions are satisfied, add the applicant to the eligible list.

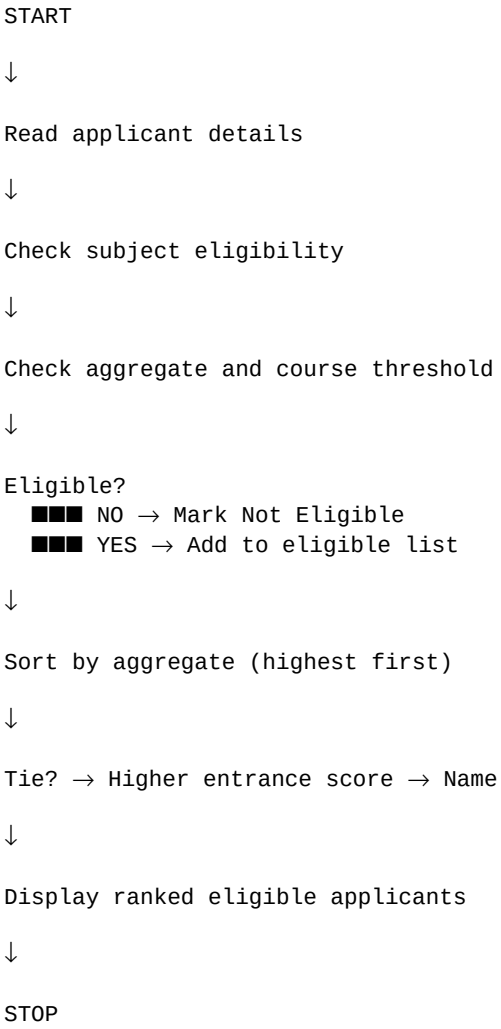
6. Sort eligible applicants by aggregate in descending order.

7. If aggregate is tied, sort by entrance score in descending order.

8. If both are tied, sort by applicant name.

9. Display the ranked eligible applicants and stop.

## 5. Flowchart


START
Read applicant details
Check subject eligibility
Check aggregate and course threshold
Eligible?
■■■ NO → Mark Not Eligible
■■■ YES → Add to eligible list
Sort by aggregate (highest first)
Tie? → Higher entrance score → Name
Display ranked eligible applicants
STOP

## 6. Pseudocode

```
BEGIN
    INPUT applicants
    eligible = empty list

    FOR each applicant:
        IF required subject marks are valid
           AND aggregate >= 70
           AND aggregate >= course threshold:
               ADD applicant to eligible

    SORT eligible by:
        aggregate descending
        entrance score descending
        name ascending

    DISPLAY ranked eligible applicants
END
```

## 7. Python Source Code

```
def admission_decision(applicants):
    eligible = []

    for a in applicants:
        if (a["math"] >= 60 and
            a["science"] >= 60 and
            a["aggregate"] >= 70 and
            a["aggregate"] >= a["threshold"]):
            eligible.append(a)

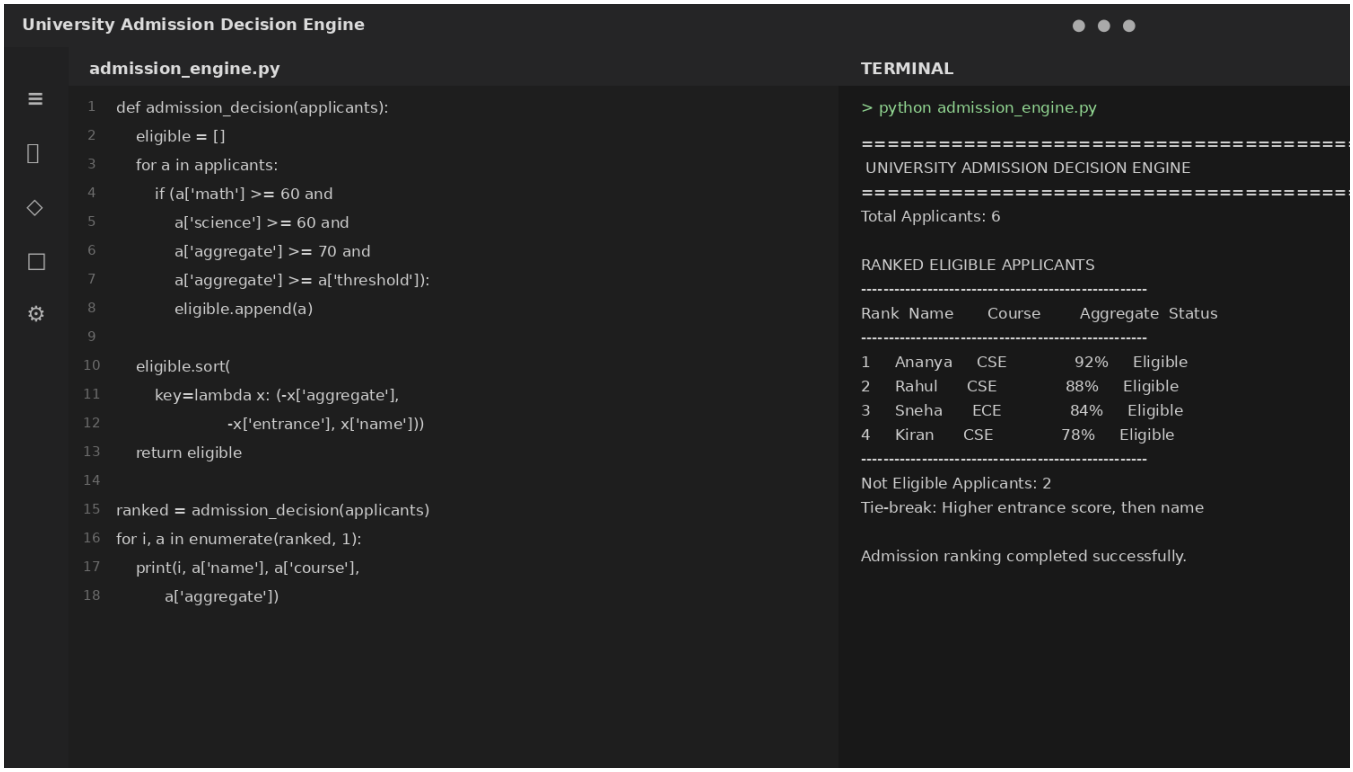
    eligible.sort(
        key=lambda x: (-x["aggregate"],
                       -x["entrance"], x["name"])
    )
    return eligible

applicants = [
    {"name":"Ananya","course":"CSE","math":92,"science":90,
     "aggregate":92,"entrance":95,"threshold":75},
    {"name":"Rahul","course":"CSE","math":88,"science":86,
     "aggregate":88,"entrance":91,"threshold":75},
    {"name":"Sneha","course":"ECE","math":84,"science":82,
     "aggregate":84,"entrance":89,"threshold":70},
    {"name":"Kiran","course":"CSE","math":78,"science":80,
     "aggregate":78,"entrance":88,"threshold":75},
    {"name":"Meena","course":"CSE","math":65,"science":68,
     "aggregate":72,"entrance":90,"threshold":75},
    {"name":"Arjun","course":"ECE","math":55,"science":75,
     "aggregate":76,"entrance":87,"threshold":70}
]

ranked = admission_decision(applicants)
for i, a in enumerate(ranked, 1):
    print(i, a["name"], a["course"], a["aggregate"])
```

## 8. Output Screenshot

VS Code-style screenshot showing the Python source and terminal output for the admission decision engine.




## 9. Test Cases

| Test | Input / Scenario | Expected Result |
|---|---|---|
| 1. Normal data | 6 applicants with mixed eligibility | 4 applicants ranked as eligible |
| 2. All eligible | Every applicant meets all rules | All applicants appear in ranked order |
| 3. Subject failure | Required subject below 60 | Applicant is Not Eligible |
| 4. Threshold failure | Aggregate below course threshold | Applicant is Not Eligible |
| 5. Aggregate tie | Same aggregate for two applicants | Higher entrance score gets higher rank |
| 6. Full tie | Same aggregate and entrance score | Name is used as final tie-break |

## 10. Conclusion

The University Admission Decision Engine checks subject eligibility, aggregate marks, and course-specific thresholds. It then ranks eligible applicants using clear tie-breaking rules. The final output provides a simple ranked list that can help universities organize admission decisions consistently.